

The PM's Guide to Testing AI Features

How AI has upended QA and made it a key PM skill

AAKASH GUPTA

OCT 06, 2025 • PAID



63



3

Share

You can't escape it. Every product team is being asked to ship [AI features](#). Fast.

And development teams are shipping AI code 3-5x faster than before using tools Cursor and Claude Code. But, their actual release cycles haven't sped up at all.

Why? Because **traditional testing approaches can't handle AI's** unique challenges.



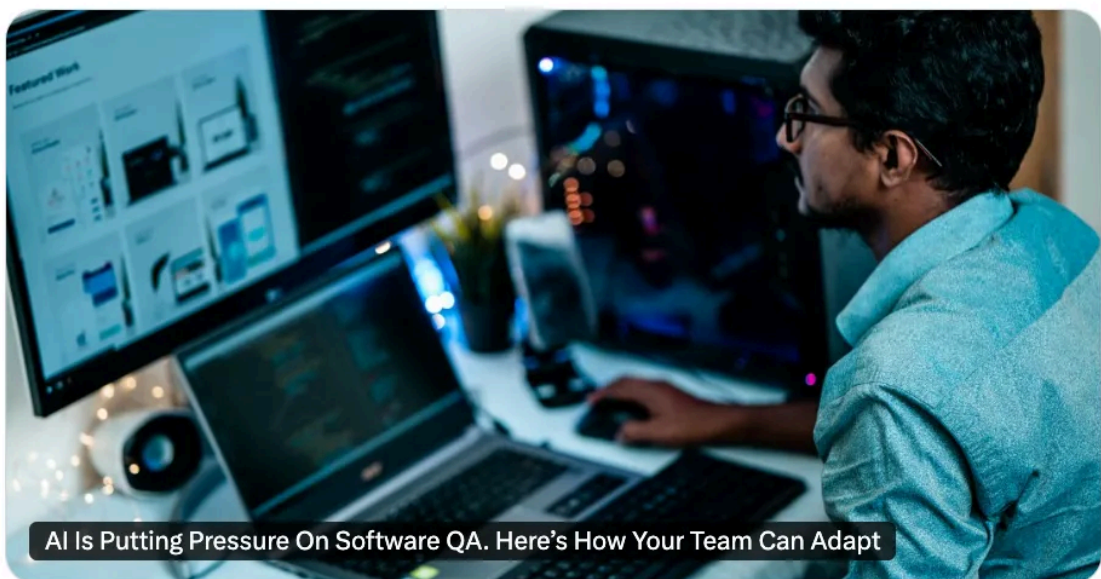
Jyoti Bansal ✓
@jyotibansalsf



Software teams, you need to get ahead of this problem:

As AI makes coding easier and faster, testing is becoming the new bottleneck — erasing productivity gains.

Here's how companies can adapt:



AI Is Putting Pressure On Software QA. Here's How Your Team Can Adapt

From linkedin.com

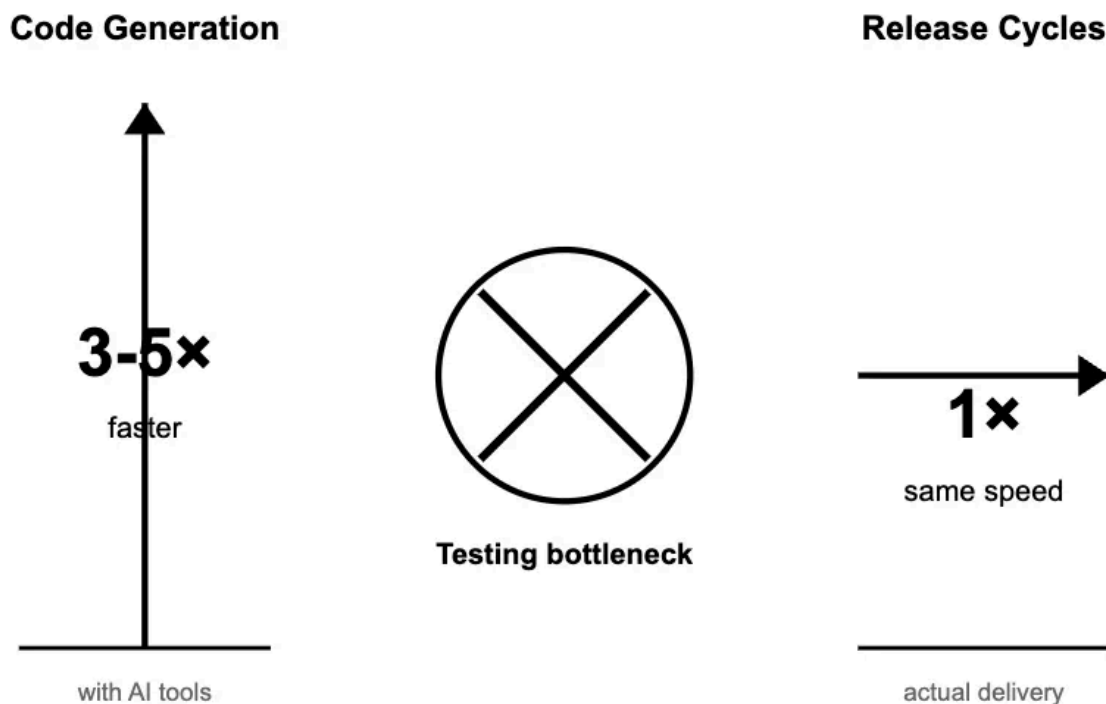
6:33 AM • Apr 25, 2024 • **3.2M** Views

Jyoti Bansal's (Founder of \$3.4B AppDynamics) [Viral LinkedIn post](#).

A Series B SaaS company I spoke with last week had their AI feature builds taking hours. Their traditional web features? 15 minutes. Same codebase, same infrastructure. The difference? AI testing complexity that their traditional workflow couldn't handle.

The result? You're promising faster AI feature delivery, but your actual release cycles are hitting unexpected bottlenecks because traditional testing workflows weren't designed for AI outputs.

The AI Velocity Paradox



Testing demands exploded. Infrastructure didn't keep up.

Today's Post

Most PMs are shipping AI features without understanding how their role in test needs to fundamentally change.

But the PMs who understand AI testing? They're shipping faster, with higher confidence, and building better products.

I'm bringing you the *first* practical guide to testing AI features as a PM:

1. Why AI testing is the new core skill every AI PM needs
2. The 4 ways AI features break traditional testing approaches
3. The AI PM's Tech Stack (yes, you have one now)
4. Step-by-step case study with real metrics
5. 3 infographics to share with your team

1. Why AI Testing is the New Core AI PM Skill

Six months ago, being an AI PM meant knowing how to write prompts and pick right model.

Today, it's about understanding how to ship AI features reliably at scale.

The landscape has shifted. Companies like Anthropic, OpenAI, and Google have **commoditized the models**. [Prompt engineering](#) is table stakes. What separates winning AI PMs from the rest?

Operational excellence in shipping AI products.

That's why I've written so much recently about [evals](#), [observability](#), and [LLM jud](#)

They are part of one bigger AI topic: **Testing**.

And the PMs who understand AI testing holistically have a massive advantage:

- They ship faster because they catch issues earlier
- They have higher confidence in releases because they understand quality
- They build better products because they design testability from day one

- They get promoted because they deliver results, not just features

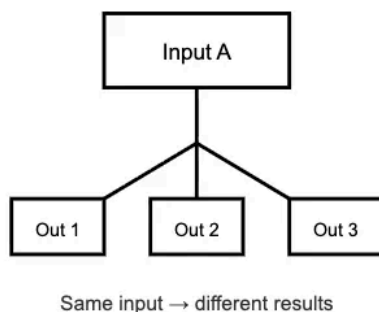
Think about it. Your CEO doesn't care if you can write a great prompt. They care if you ship AI features that work, scale, and don't break the core product.

2. The 4 Ways AI Features Break Traditional Testing

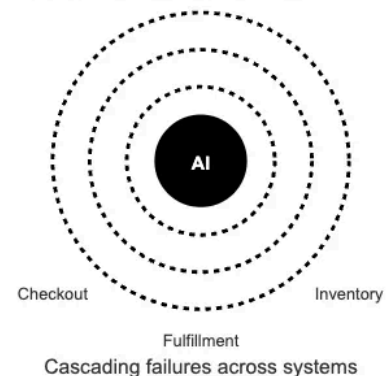
Traditional features are predictable. Given input A, you get output B. Every time AI features? Not so much. Here are the 4 fundamental ways AI breaks traditional testing:

4 Ways AI Breaks Traditional Testing

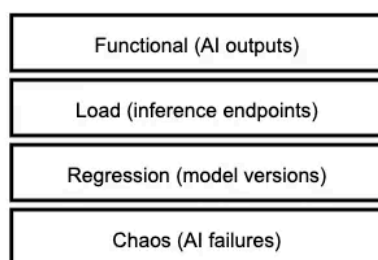
1. Non-Deterministic Outputs



2. Broader Blast Radius

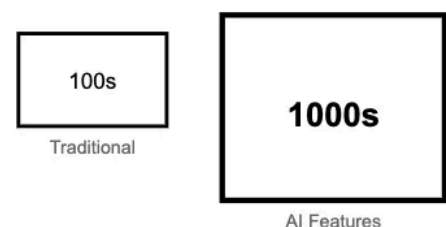


3. New Testing Categories



Tests you've never run before

4. Scale Requirements



10x more test scenarios

Your traditional test suite catches only 30% of AI issues

1. Non-deterministic outputs mean the same input can produce different results. A chatbot might give a perfect answer on Tuesday and complete nonsense on Wednesday. Both responses pass your existing unit tests.

2. Broader blast radius across systems. Your AI chat feature doesn't just affect the chat module. It can break checkout flows in unexpected ways. When an AI recommendation engine starts suggesting out-of-stock items, it cascades through inventory, fulfillment, and customer service systems.

3. New testing categories you've never had to think about:

- Functional testing for AI outputs (not just code paths)
- Load testing for inference endpoints under real traffic
- Regression testing across model versions
- Chaos testing for when AI systems fail (and they will)

4. Scale requirements that break traditional approaches. AI features generate thousands of test scenarios. Your old test suite was designed for hundreds.

The result? Your traditional test suite catches maybe a fraction of the issues that matter for AI features.

As [Dmitry Fonarev](#), CEO of [Testkube](#), puts it:

"The side effect of AI generating more code is that the overall testing of software applications becomes more critical. We can't just run a small set of tests against a subsystem anymore. We need to validate what an API change may have on second, third, and fourth order systems behind it."

Translation for PMs: Your tests must anticipate model drift, not just code changes. Otherwise you're stuck managing regressions you didn't cause.

For a deeper dive into the 7 testing categories you must know, check out [my complete AI testing taxonomy guide](#):

From Problem to Solution

You understand the problem. Now get the solution:

- **3-element testing framework** you can implement this week
- **The AI PM's tech stack and architecture guide** with ready-to-share infogra
- **Real case study**: How one team cut builds from 4 hours to 20 minutes
- **Conversation framework** for every team you work with

*The AI PMs who master testing infrastructure will dominate. The ones who don't will m
firefights instead of building products.*

3. Framework: How to Think About Testing AI as PM

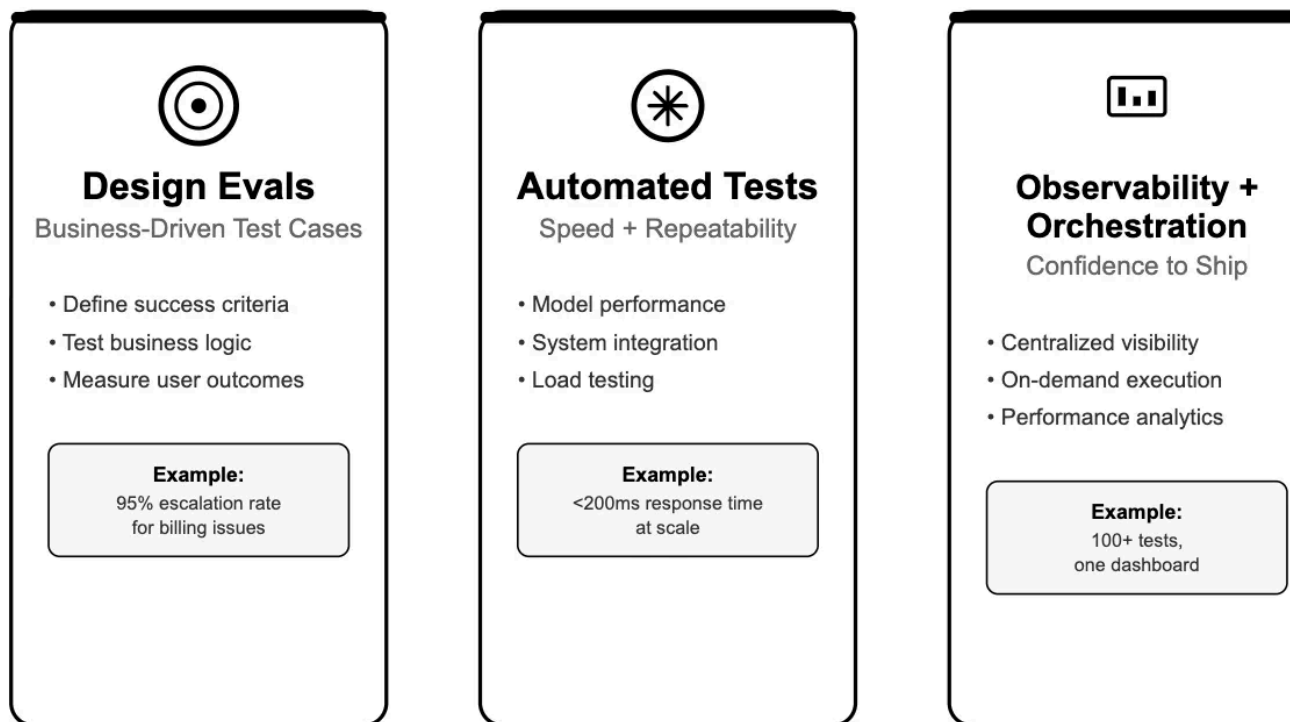
[Ole Lensmar](#), CTO and Co-founder of Testkube, points out a critical challenge:

*“Both what you build is non-deterministic, and the model you use is non-determinist
Even if you feel what you’re adding to the model is deterministic, the underlying mod
not be. When GPT-4 changes to GPT-5, people may want their GPT-4 back because
change between models is very fundamental.”*

So how do you actually approach AI testing as a PM? Three elements:

The AI Testing Framework for PMs

AI PMs must think in all three layers simultaneously



The three-layer approach that scales with AI development

Testing Element 1 - Design Evals (Business-Driven Test Cases)

These are business logic tests for AI outputs.

Example: Your AI support chatbot should:

- Escalate billing issues to human agents 95% of the time
- Never provide refunds over \$500 without approval
- Maintain customer satisfaction scores above 4.0/5

Design these test cases alongside your feature specs, not after.

Testing Element 2 - Automated Tests (For Speed + Repeatability)

Your AI features need three types of automated testing:

1. **Model performance tests:** Does the AI perform within acceptable bounds?
2. **Integration tests:** Do AI features work with existing systems?
3. **Load tests:** Can AI endpoints handle production traffic?

Testing Element 3 - Observability + Orchestration (Confidence to Ship)

This is where most teams fail. You need:

- **Centralized test management:** One place to see all test results
- **On-demand test execution:** Run tests when you need them, not just during launch
- **Performance analytics:** Track test trends over time
- **Workflow orchestration:** Run different test suites for different scenarios

Without orchestration, you're flying blind. You can't answer basic questions like "Are we ready to ship?" or "What broke in the last release?"

AI Prompt: Generate Your AI Feature Test Plan

Use this prompt to quickly create comprehensive test cases for your AI feature. Just replace the bracketed sections with your specifics:

I'm building an AI feature that [describe core functionality].

Help me create a comprehensive test plan using this framework:

1. BUSINESS-DRIVEN TEST CASES (Evals)

- What are the top 5 critical business outcomes this AI feature must achieve?
- What specific failure modes would cause the most user/business harm?
- What quantitative thresholds should trigger alerts? (e.g., accuracy >90%, latency <2s)

2. AUTOMATED TEST CATEGORIES

For each category below, suggest 3-5 specific test scenarios:

- Model performance tests
- Integration tests with existing systems
- Load/scale tests

- Edge cases and failure scenarios

3. TESTING GAPS

Based on my current setup: [describe your current testing approach]

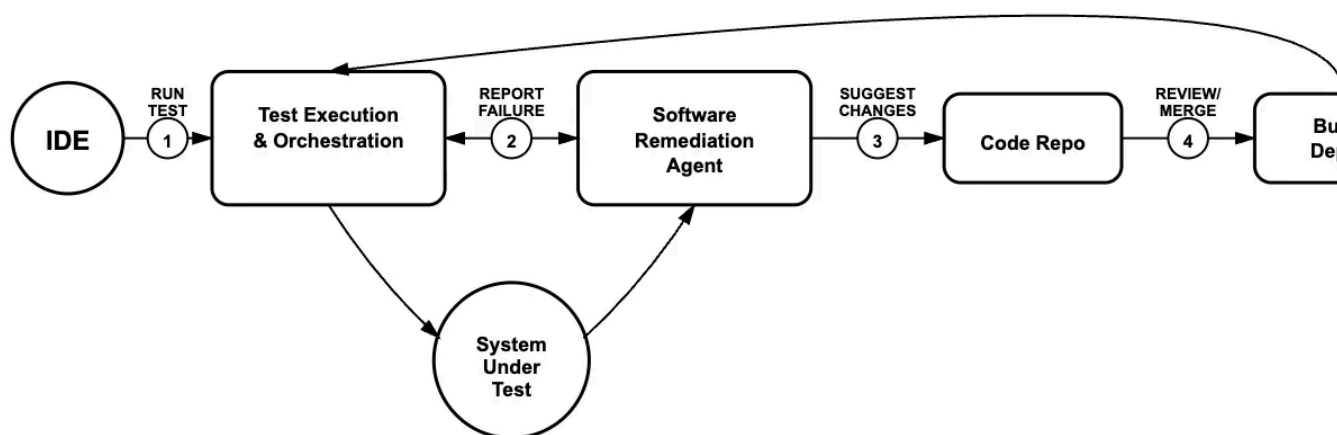
- What critical test categories am I missing?
- Which tests should be prioritized based on risk?
- What would a "minimum viable test suite" look like for launch?

Output the results in a format I can share directly with my engineering team.

Now here's where it gets interesting. The future of testing isn't just about running tests faster. It's about making testing intelligent.

The Goal: AI Testing Self-Healing Loop

Modern testing platforms like Testkube are starting to integrate AI in ways that create closed-loop systems. Here's what you want to build:



Take the scenario that *your AI recommendation engine starts suggesting out-of-stock*

The traditional approach would be:

Test fails → Engineer gets alert → Manual investigation → Code fix → Manual re

The Self-healing approach is:

1. **AI analyzes the failure:** "Inventory API returning stale data, affecting recommendation accuracy"
2. **Suggests specific fix:** "Update inventory refresh interval from 1 hour to 15 minutes in recommendations.config"
3. **Auto-creates pull request** with the proposed change
4. **Re-runs tests** to validate fix worked

This takes **time to resolution** from 2 Hours —> 15 minutes

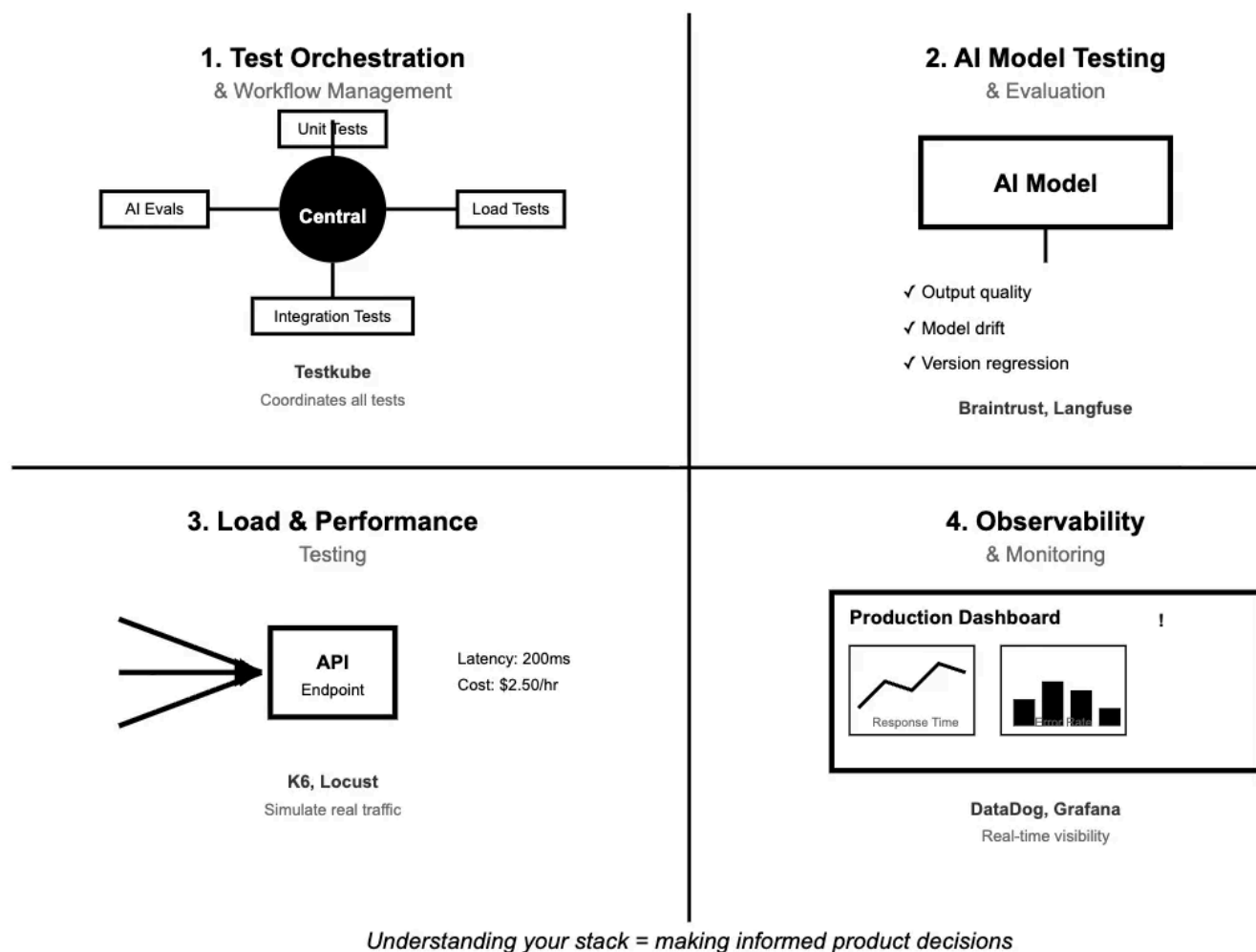
And transforms testing from a bottleneck into an accelerant for AI development teams. Instead of waiting for humans to debug AI system interactions, the testing infrastructure becomes self-correcting.

4. The AI PM's Tech Stack (Yes, You Have One Now)

Understanding the framework is one thing. Knowing what tools power it is another.

Just like you evaluate analytics platforms or customer feedback tools, you now need to understand four categories of testing infrastructure:

The AI PM's Tech Stack: 4 Categories



Category 1 - Test Orchestration

This is your central nervous system for testing. It's what coordinates all your diff tests, decides what runs when, and gives you visibility into results.

Think of it like Jira for testing. Just as you wouldn't manage a product backlog in threads, you can't manage complex AI testing in bash scripts.

You'll need

- **Workflow definition:** "Run unit tests first, then integration tests, then load t
- **Parallel execution:** Run 100 tests simultaneously instead of sequentially
- **Conditional logic:** "Only run expensive load tests if code changed affects AF endpoints"

Example tools: Testkube (Kubernetes-native test orchestration), Argo Workflows, Jenkins X

Category 2 - AI Model Testing

These tools specifically test AI model outputs, not just code correctness.

They help you do:

- Regression testing across model versions
- Evaluation frameworks for LLM outputs
- Prompt testing and optimization

Example tools: Braintrust, Langfuse, PromptLayer, Weights & Biases, Arize

Category 3 - Load & Performance

AI endpoints are expensive and slow. You need to know they'll perform under real world traffic before you ship.

Your CFO will thank you for this one. Load testing reveals cost explosions before they hit your P&L. One team discovered their AI feature would cost \$50K/month at scale. Load testing revealed this before launch, not after.

Example tools: K6, Locust, Gatling, Artillery

Category 4 - Observability

Once your AI features are in production, you need to know what's actually happening.

Key capabilities you need:

1. Real-time AI performance monitoring
2. User interaction analytics
3. Cost tracking per feature
4. Model drift detection

Example tools: DataDog, New Relic, Grafana, OpenTelemetry

Model drift is silent. Your AI can start degrading gradually, and without observation you won't know until users start churning. Observability gives you early warning systems.

You don't need to configure these tools yourself. But you need to:

- **Understand what's in your stack:** “We use Testkube for orchestration, Brain for eval, K6 for load testing”
- **Know the gaps:** “We have no automated model evaluation framework”
- **Make informed tradeoffs:** “We should invest in better load testing before we release this feature to all users”
- **Speak the language:** When your DevOps team says “our test orchestration can handle this,” you understand what they mean

Your tech stack is now part of your product stack. Treat it accordingly.

5. Case Study: Manual vs Orchestrated Testing

This is what happens when you ignore testing infrastructure.

A Series B SaaS company built an AI-powered analytics feature. Here's how their testing worked:

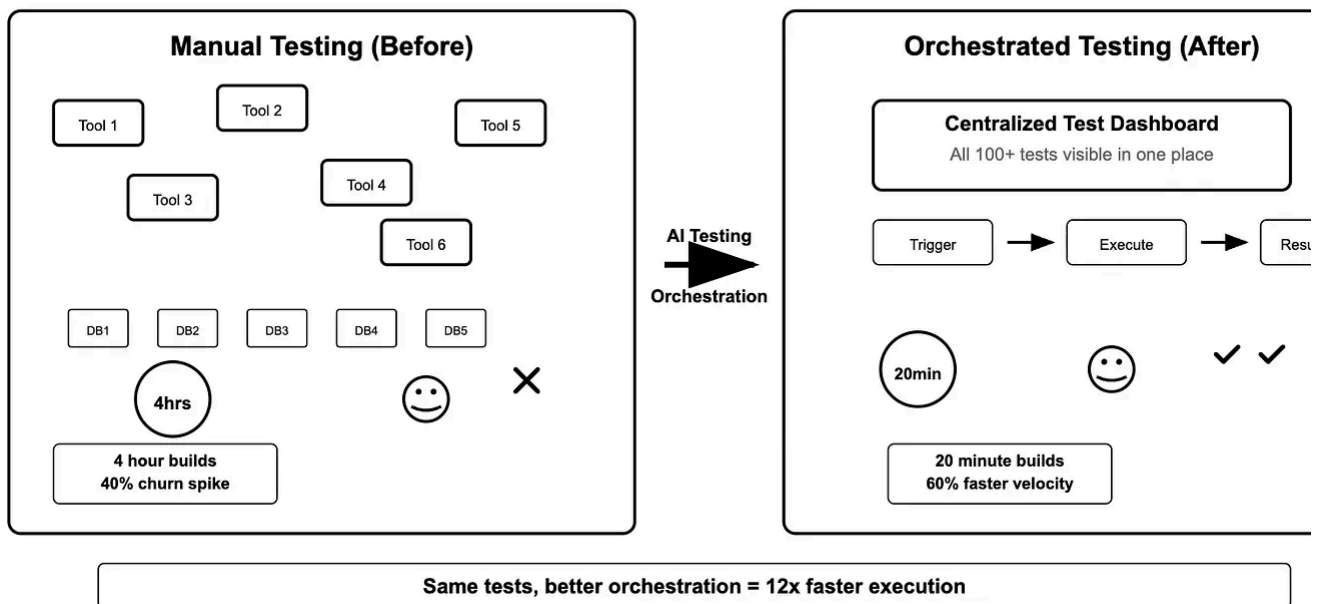
- 30 different test scripts across 6 different tools
- Tests only ran during builds (2-3 times per day)
- No centralized results - engineers checked 5 different dashboards
- 4-hour build times with all tests running sequentially
- Manual triage for every test failure

They shipped a major bug to production because load tests weren't running consistently. Customer churn spiked 40% before they caught it.

Their VP of Product told me: “We thought testing was an engineering problem. We were wrong. It became a business problem.”

The fix? Creating this new AI PM tech stack:

Before vs After: Testing Architecture



They used tools and orchestration to create:

1. **Centralized test catalog** - all 100+ tests visible in one dashboard
2. **On-demand execution** - PMs could trigger specific test suites anytime
3. **Parallel execution** - build times dropped from 4 hours to 20 minutes
4. **Automated triage** - test failures categorized and routed automatically
5. **Performance analytics** - they could track test trends and optimize over time

The ROI was immediate:

- 75% reduction in build times (4 hours → 20 minutes)
- 40% fewer production bugs in month one

- \$200K annual savings in developer time previously spent waiting for builds

Most importantly, they shifted from reactive firefighting to proactive feature development.

Thanks to the Testkube team for creating that connection and partnering on this piece.

6. The PM-DevOps Relationship

By now you realize AI testing isn't something you hand off to DevOps. It's a collaborative product effort.

The best AI PMs treat DevOps as a core product partner, not a service team.

[Use my PM-DevOps communication template](#) to begin a conversation with them

Then, work with them across the development lifecycle:

Before Development: Set Testing Requirements Together

Don't spec features then ask "can we test this?" Build testability into requirements from day one.

- **PM:** Define business-critical test cases (your evals from Element 1)
- **DevOps:** Assess infrastructure needs and constraints
- **Together:** Agree on what "ready to ship" means

Example: Planning an AI recommendation engine? You define 90% relevance score requirement. DevOps determines you need load testing for 10K concurrent users align before writing specs.

During Development: Maintain Test Coverage Visibility

PMs should ask: “What’s our test coverage for core evals?” and “Where are repeated failures coming from?”

Don’t disappear until demo day. Stay engaged on what’s tested and what’s not.

Post-Launch: Close the Improvement Loop

You guys should come back together.

- **PM:** Feed production insights back into eval criteria
- **DevOps:** Maintain and optimize test infrastructure
- **Together:** Monthly review of test effectiveness

When users abandon AI outputs with certain patterns, add it as an eval. DevOps it to automated tests.

This is **how you sustainably improve your AI systems**.

Takeaway: Checklist

If you take away one thing, it's this: You don't need to run tests, but you need to **know what to test, when, and why**.

Here’s your checklist to do so:

AI PM's Testing Checklist

Key questions before shipping

Quality & Reliability

- ☐ "What's our target accuracy/quality bar and are we hitting it?"
- ☐ "Have we tested edge cases and failure modes?"
- ☐ "Can we detect and handle hallucinations?"

User Experience

- ☐ "How does the AI degrade gracefully when it fails?"
- ☐ "Are we testing real user inputs, not just synthetic ones?"
- ☐ "Can users give feedback when AI gets it wrong?"

Safety & Deployment

- ☐ "Have we tested for bias, toxicity, and harmful outputs?"
- ☐ "Do we have kill switches and gradual rollout?"
- ☐ "What metrics tell us if this is working post-launch?"
- ☐ "Are we monitoring costs and can we scale sustainably?"

Final Words

In the AI era, testing is no longer QA. It is product infrastructure.

The PMs who master it will out-ship everyone else.



Recommend Product Growth to your readers

Deep dives into succeeding as a PM, product leadership, and how to get your next PM job.

Recommend



63 Likes · 3 Restacks

← Previous

Next →

Discussion about this post

Comments

Restacks



Write a comment...

© 2026 Aakash Gupta · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture